

Arbeiten mit dem Clustersystem der LUH am Institut für Produktionswirtschaft

Kolloquium SoSe 2026

Timo Helfers

Leibniz Universität Hannover
Wirtschaftswissenschaftliche Fakultät
Institut für Produktionswirtschaft

05. Mai 2026



① Cluster-Dokumentation

② Cluster einrichten

③ Cluster-Runs

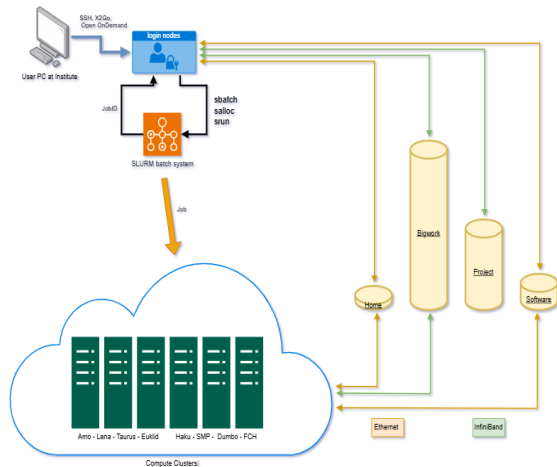
① Cluster-Dokumentation

② Cluster einrichten

③ Cluster-Runs

- Alles Wichtige zum Cluster hier zu finden:
`https://docs.cluster.uni-hannover.de/doku.php?id=start`
- Im Folgenden kurze Zusammenfassung des Aufbaus und Idee des Clusters

- Zugang via Browser oder Code-Editor (mit SSH)
- Aufgeteilt in vier „file systems“
 - Home
 - Bigwork
 - Project
 - Software



Grundsätzlicher Aufbau des Clusters

Quelle: https://docs.cluster.uni-hannover.de/doku.php/guide/about_the_cluster_system

- Soft limit = 100 GB, Hard limit = 1 TB, Verbindung = Schnell (InfiniBand)
- Primärer Ort für tägliche Arbeit
- Bietet vergleichsweise hohes Speichervolumen
- Standardort für Berechnungen
- Auf allen Knoten des Clusters verfügbar

⇒ Hier Git-Projekte ablegen und Cluster-Runs starten, sowie die Ergebnisse abspeichern

- Soft limit = 10 TB, Hard limit = 12 TB, Verbindung = Moderat (InfiniBand)
- Langfristige Speicherung von Daten und Projekten
- Gemeinsame Nutzung für alle Projektmitglieder
- Keine Verbindung zu den Compute-Nodes, Berechnungen hier also nicht möglich
- Doppeltes Abspeichern hier ist kein vollwertiges Backup
- Regelmäßiges Abspeichern von \$BIGWORK auf \$PROJECT dennoch sinnvoll

⇒ Als eine Art Backup/Datenaustausch-System nutzbar auf das alle Institutsmitglieder Zugriff haben

- Soft limit = 10 GB, Hard limit = 12 GB, Verbindung = Langsam (Ethernet)
- Für kleine wichtige Konfigurationsdateien, Skripte und Vorlagen gedacht
- „Don't do big work in your home“
- Überlastung durch Job-Daten beeinträchtigt alle Nutzer
- Nicht für conda oder pip verwenden
- Einziges Verzeichnis mit täglicher Datensicherung
- Bei Quota-Überschreitung nicht wahllos Dateien löschen

⇒ Eigentlich nicht mehr anfassen, nachdem alles konfiguriert ist

- Soft limit = 50 GB, Hard limit = 100 GB, Verbindung = Langsam (Ethernet)
- Gruppenweite Installationen, bspw. conda
- Zugriffsrechte können präzise gesteuert werden
- Wie \$HOME langsam und nicht für Rechenjobs geeignet
- Hält \$HOME und \$BIGWORK sauber

⇒ Bisher bei uns nicht in Gebrauch

Charakteristika der verschiedenen Partitionen

Quelle: https://docs.cluster.uni-hannover.de/doku.php/resources/computing_hardware

Partition	Nodes	CPUs	Cores /Node	Cores Total	(useable) Memory /Node (MB)	Memory Total (GB)	Gflops /Core ¹⁾	Local Disk /Node (GB)	Node Interconnect
mpp.share	27	2x AMD EPYC 9534	128	3456	500.000	13.500	80	800 (NVMe)	InfiniBand NDR, 200 Gbs
mpp.single	10	2x AMD EPYC 9534	128	1280	500.000	5.000	80	800 (NVMe)	InfiniBand NDR, 200 Gbs
amo	80	2x Intel Cascade Lake Xeon Gold 6230N	40	3200	180.000	15360	75	400 (SSD)	InfiniBand HDR, 100 Gbs
taurus	24	2x Intel Skylake Xeon Gold 6130	32	768	120.000	3072	67	500 (HDD)	InfiniBand EDR, 100 Gbs
haku	20	2x Intel Broadwell Xeon E5-2620 v4	16	320	60.000	1280	34	80 (SSD)	InfiniBand FDR, 40 Gbs
lena	80	2x Intel Haswell Xeon E5-2630 v3	16	1280	60.000	5120	38	180 (SSD)	InfiniBand QDR, 40 Gbs

Charakteristika der verschiedenen Partitionen

Quelle: https://docs.cluster.uni-hannover.de/doku.php/guide/slurm_usage_guide

Partition	Default/Max Job Runtime	Max Nodes Per Job	Max CPUs per User	Default/Max Memory per CPU	Node Shared
mpp.single	1/48 hr		512	3900/3900 MB	no
mpp.share	1/48 hr			2000/4096 MB	yes
amo	24/200 hr			4000/5120 MB	yes
taurus	24/200 hr			2000/4096 MB	yes
lena	24/200 hr			3800/4096 MB	yes
haku	24/200 hr			3800/4096 MB	yes
smp	12/200 hr	1	512	4000/8000 MB	yes
gpu	1/48 hr	1		2000/8000 MB	yes
vis	1/6 hr	1	8	2000/4500 MB	yes

2 Schnelleres Einplanen eigener Jobs möglich

- Scheduler passt Forderungen so an, dass sie aufgehen
 - Forderung: 40 GB, 1 Kern $\Rightarrow$ Reservierung: 40 GB, 10 Kerne
 - Je mehr angefordert wird, desto mehr reservieren wir
 - Statt 4100 MB nur 4096 MB anfordern $\Rightarrow$ Einsparung eines Kerns
- chnelleres Einplanen eigener Jobs möglich
- Dazwischenquetschen seltener möglich bei hohen Ressourcenforderungen
 $\Rightarrow$ Einplanen ist schwieriger und eher später
 - Hinreichende Menge an Ressourcen anfordern nicht mehr
 - Mehr Kerne heißt nicht schneller, nur sinnvoll wenn Parallelisierung möglich

Partition	amo
State	RUNNING
Reason	None
Total Nodes	10
Node List	amo-n005, amo-n018, amo-n029, amo-n030, amo-n031, amo-n032, amo-n059, amo-n060, amo-n067, amo-n070
Total CPUs	100
Time Limit	8-08:00:00
Time Used	3-15:38:14
Start Time	2026-04-25 17:20:49
End Time	2026-05-04 01:20:49
Memory	3072M

Abbildung: Möglicherweise ein Fall von „Je mehr ich anfordere, desto besser läuft mein Code“



① Cluster-Dokumentation

② Cluster einrichten

③ Cluster-Runs

- ① Webbrowser: `https://login.cluster.uni-hannover.de/auth/login.html`
 - Erreichbar mit (W)LAN- oder VPN-Verbindung
- ② Via SSH integriert in eigenem VS-Code-Editor
 - Erreichbar mit (W)LAN- oder VPN-Verbindung
 - Möglich, wird aber nicht empfohlen

- 1 Anmelden im Webbrowser:
`https://login.cluster.uni-hannover.de/auth/login.html`
- 2 Navigieren zu \$BIGWORK und Öffnen eines neuen Terminals
- 3 Befehl eingeben: `ssh-keygen -t ed25519 -C "Beispiel-Name"`
- 4 Navigiere zu \$HOME/.ssh, öffne **id_ed25519.pub** und kopiere den gesamten Inhalt
- 5 Öffne die Preferences in Gitlab (`https://gitlab.uni-hannover.de/users/sign_in`) und navigiere zu SSH Keys
- 6 Klicke Add new key, füge den kopierten key ein, Namen geben und Ablaufdatum setzen
→ Add key
- 7 SSH-Verbindung testen im Terminal des Webbrowsers mit Befehl:
`ssh -T git@gitlab.uni-hannover.de`
⇒ Welcome to GitLab, @Benutzername!

- `https://gitlab.uni-hannover.de/timohelfers/example\_cluster`
- Projekt via SSH klonen



- 1 Generiere Key für lokalen Rechner (falls noch nicht vorhanden):
`ssh-keygen -t ed25519 -C „<deine_email@example.com>“`
- 2 Füge den Key der Liste an autorisierten Keys im Cluster-Account hinzu (Git-Bash):
`ssh-copy-id -i ~/.ssh/id_ed25519.pub <Benutzername>@login.cluster.uni-hannover.de`
- 3 VS Code öffnen und Extension „Remote-SSH“ von Microsoft installieren
- 4 In Extension neue Verbindung via „New Remote“ hinzufügen
- 5 In Pop-Up am oberen Bildschirmrand eingeben:
`<Benutzername>@login.cluster.uni-hannover.de`
- 6 (In config-file Zeile hinzufügen: `IdentityFile ~/.ssh/id_ed25519`)



- Auf Cluster mit virtuellen Environments arbeiten
- Bisher mit Miniforge3 (conda)
- Bringt einige Schwierigkeiten mit sich:
 - Manuell Packages hinzufügen
 - Default-Pfad zum Speichern der Environments anpassen
 - Mehrere Environments für verschiedene Projekte unübersichtlich
⇒ Fehleranfällig, aufwendig und nervig

- Letzte Woche uv kennengelernt
- uv ist auf dem Cluster nutzbar
- Vorteile:
 - Packages automatisch geladen bei Durchlauf über „uv sync“ mit aktueller .toml-Datei
 - .venv in jeweiligem Projektpfad gespeichert (**nicht in Git!!!**)
⇒ Wenn Git-Projekt und damit die .toml-Datei aktuell ist, gibt es keine Probleme

- Kurze Live-Demo



① Cluster-Dokumentation

② Cluster einrichten

③ Cluster-Runs

- Grundstruktur:
 - 1 (build.sh) bei C++
 - 2 queue.sh
 - 3 job.sh
- (Wir starten build.sh führen einen Build aus und rufen queue.sh auf)
- Wir starten die queue.sh
- Ruft iterativ job.sh mit verschiedenen Argumenten und Ressourcenanforderungen auf
- job.sh startet die main-Datei mit Argumenten unter eingestellten Ressourcenanforderungen
 - ⇒ Zeigen im geklonten Beispielprojekt

- Befehle:
 - Skript ausführen
`sbatch <SKRIPT_NAME.sh>`
 - Warteschlange des Users anschauen (auch aktive Jobs enthalten)
`queue -u nhul<NAME>`
 - Alle Jobs des Users abbrechen
`scancel -u nhul<NAME>`
 - Details über Partition anzeigen lassen
`scontrol show partition <PARTITION>`
 - Speicherauslastung der file systems anzeigen
`checkquota`
 - und viele weitere zu finden in der Dokumentation

- Maximal 500 Jobs gleichzeitig in Warteschlange
- Bis zu 64 Jobs eines Users laufen gleichzeitig
- Besonders viele CPU-Stunden in einem Quartal verringern die Priorität des Users

Vielen Dank für Ihre Aufmerksamkeit!